

Graph Spanner Approximations in Practice: Engineering and Benchmarking Sparse Distance-Preserving Subgraphs

Sheikh Muhammad Muneeb Abdul Moiz Ahmed Moosa Owais¹

¹Habib University, Karachi, {ms08914, aa08811, mo08952}@est.habib.edu.pk

11 March 2026

Abstract

This proposal outlines the implementation and empirical evaluation of two graph spanner construction algorithms, as studied by Chimani and Stutzenstein (2022) in ESA 2022. A multiplicative α -spanner is a sparse subgraph that preserves all pairwise distances within a factor of α . We implement the classical Greedy algorithm and the randomized Baswana–Sen clustering algorithm, evaluating them on synthetic and real-world networks with respect to running time, sparseness, lightness, and effective stretch.

1 Introduction and Problem Selection

The chosen project follows Option A: Paper-Based Implementation. We have selected the paper “Spanner Approximations in Practice” published in the 30th Annual European Symposium on Algorithms (ESA 2022), Track B (Engineering and Applications) [Chimani and Stutzenstein, 2022].

A *multiplicative α -spanner* H of a weighted undirected graph $G = (V, E)$ is a subgraph $H \subseteq G$ that satisfies $d_H(u, v) \leq \alpha \cdot d_G(u, v)$ for all $u, v \in V$, where d_G denotes shortest-path distance. The goal is to construct such an H with as few edges as possible. Spanners have direct applications in routing, network design, distributed computing, and approximate distance oracles, as surveyed in Chimani and Stutzenstein [2022].

Despite the existence of many spanner algorithms with strong theoretical guarantees, no systematic experimental comparison existed prior to Chimani and Stutzenstein [2022]. Their paper fills this gap by implementing and benchmarking multiple algorithms on diverse graph families. We focus on the two most accessible algorithms in that study: the Greedy spanner and the Baswana–Sen (BS) clustering algorithm.

2 Brief Summary of Research Contribution

The primary contribution of Chimani and Stutzenstein [2022] is the first comprehensive experimental study of multiplicative spanner approximation algorithms. The paper implements and evaluates five algorithms across three graph libraries (Random, TSPLIB, Steinlib), comparing them on sparseness, lightness, effective stretch, and running time. A key finding is that the theoretically optimal Greedy algorithm consistently produces the sparsest spanners in practice, while the Baswana–Sen clustering approach achieves the best running time, solving all random instances in under 0.6 seconds.

2.1 Mathematical Formulation

Let $G = (V, E, w)$ be a weighted undirected graph with $n = |V|$ vertices and $m = |E|$ edges. For a stretch factor $\alpha \geq 1$ and a subgraph $H = (V, E_H)$ with $E_H \subseteq E$, define:

$$d_H(u, v) \leq \alpha \cdot d_G(u, v) \quad \forall u, v \in V \quad (1)$$

The *sparseness* of H is $|E_H|/n$, and its *lightness* is $w(E_H)/w(\text{MST}(G))$. The Greedy algorithm is known to guarantee $|E_H| = O(n^{1+1/k})$ edges for stretch $\alpha = 2k - 1$, while the Baswana–Sen algorithm produces the same $(2k - 1)$ -spanner in expected $O(km)$ time via randomized cluster sampling; both results are used as baselines in Chimani and Stutzenstein [2022].

3 Feasibility and Relevance Justification

The spanner problem is a canonical subgraph optimization problem with a clean stretch–sparseness trade-off. The two algorithms we implement require only standard data structures: priority queues (Greedy) and adjacency lists (Baswana–Sen). Both are expressible in Python using NetworkX for graph primitives and

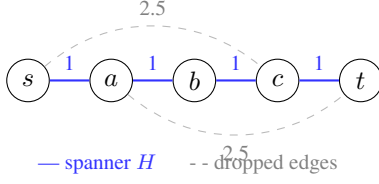


Figure 1: A 3-spanner of a 5-node graph constructed by the Greedy algorithm ($k = 2$, $\alpha = 3$). Edges are processed in non-decreasing weight order: the four path edges (weight 1) are added first, building $H = s-a-b-c-t$. The two chords (weight 2.5) are then considered and dropped because $d_H(s, c) = 3 \leq 3 \times 2.5 = 7.5$ and $d_H(a, t) = 3 \leq 3 \times 2.5 = 7.5$.

shortest-path oracles, with no external solver dependencies. The correctness criterion—verifying inequality (1) via all-pairs shortest paths on small instances—admits a straightforward oracle implementation.

Figure 1 illustrates a 3-spanner of a five-vertex graph: two edges (gray dashed) are safely omitted because paths through the spanner already satisfy the stretch constraint.

4 Implementation and Experiment Plan

Implementation proceeds in three phases:

1. **Greedy Spanner:** Sort edges by weight; add each edge if the current spanner does not already provide a $d \leq \alpha w(u, v)$ path, as described in Chimani and Stutzenstein [2022].
2. **Baswana–Sen Spanner:** Execute $k-1$ rounds of cluster sampling to build a $(2k-1)$ -spanner in expected $O(km)$ time, as described in Chimani and Stutzenstein [2022].
3. **Evaluation Suite:** Measure and compare both algorithms on all benchmark instances from Table 1.

Algorithm 1 describes the Greedy construction. Algorithm 2 describes the Baswana–Sen procedure.

4.1 Algorithmic Approach

For the Greedy spanner, the inner distance query runs Dijkstra on the current partial spanner H . Because H grows monotonically and early iterations involve a sparse H , this is fast in practice. The Baswana–Sen algorithm requires only BFS traversals and adjacency-list lookups, achieving its $O(km)$ bound without any global distance computation.

Algorithm 1 Greedy $(2k-1)$ -Spanner

Require: Graph $G = (V, E, w)$, integer $k \geq 1$
Ensure: Subgraph H with stretch $\alpha = 2k - 1$

- 1: Sort E by w in non-decreasing order
- 2: Initialize $H \leftarrow (V, \emptyset)$
- 3: **for** each edge $(u, v) \in E$ **do**
- 4: $d \leftarrow$ shortest-path dist. from u to v in H
- 5: **if** $d > (2k - 1) \cdot w(u, v)$ **then**
- 6: Add (u, v) to H
- 7: **end if**
- 8: **end for**
- 9: **return** H

Algorithm 2 Baswana–Sen $(2k-1)$ -Spanner

Require: Graph $G = (V, E, w)$, integer $k \geq 1$
Ensure: Spanner H with expected $O(n^{1+1/k})$ edges

- 1: $E_H \leftarrow \emptyset$; $\mathcal{C} \leftarrow \{\{v\} : v \in V\}$
- 2: **for** $i = 1$ **to** $k - 1$ **do**
- 3: Sample each cluster centre into R_i w.p. $n^{-1/k}$
- 4: **for** each $v \notin R_i$ **do**
- 5: **if** v has a neighbour in a cluster centred at R_i **then**
- 6: Add lightest such edge to E_H ; join that cluster
- 7: **else**
- 8: Add all edges from v to its current cluster to E_H
- 9: Remove v from $\mathcal{C}$
- 10: **end if**
- 11: **end for**
- 12: $\mathcal{C} \leftarrow$ clusters centred at R_i
- 13: **end for**
- 14: Add all edges within remaining clusters to E_H
- 15: **return** $H = (V, E_H)$

4.2 Evaluation Metrics

We measure five quantities for each (algorithm, instance, k) triple:

1. **Sparseness:** $|E_H|/n$ (edges per vertex).
2. **Lightness:** $w(E_H) / w(\text{MST}(G))$.
3. **Effective stretch:** For small instances ($n \leq 500$), exact $\max_{u \neq v} d_H(u, v) / d_G(u, v)$ via all-pairs shortest paths. For larger instances, we sample 1,000 random source–target pairs and report the maximum observed ratio as an approximate lower bound on the true stretch.
4. **Peak memory:** Measured via Python’s `tracemalloc` module at the point of max-

Family	Source	$ V $ range	N
Random (ER)	Synth.	100–5000	30
TSPLIB	TSPLIB	up to 7,397	111
Steinlib	Steinlib	up to $\geq 20,000$	96
Facebook	SNAP	4,039	1
Gnutella	SNAP	6,301	1

Table 1: Benchmark instances replicating Chimani and Stutzenstein [2022] plus SNAP extensions. The two largest TSPLIB instances ($n = 5,915$ and $n = 7,397$) are excluded from Greedy runs due to runtime constraints, matching the original study.

imum allocation during spanner construction.

5. **Running time:** Wall-clock time averaged over 5 runs per instance.

4.3 Anticipated Technical Challenges

Greedy bottleneck. Each edge in the Greedy algorithm triggers a Dijkstra call on the growing H , giving $O(m^2 \log n)$ worst-case time for dense graphs. We cap Greedy runs at $n \leq 250$ for TSPLIB and Steinlib, consistent with Chimani and Stutzenstein [2022], and profile per-instance runtimes to identify the practical threshold.

Randomness in Baswana–Sen. The BS algorithm is randomized; spanner size varies across runs. We will report means and standard deviations across 10 independent runs per instance, following the repeated-trial methodology of Chimani and Stutzenstein [2022].

5 Resources Declaration

Python is the primary language. Core libraries: NetworkX for graph operations and shortest-path oracles; NumPy and Matplotlib for data aggregation and visualization. Real-world graphs are sourced from the SNAP repository. Experiments run on personal workstations with standard CPU configurations.

References

Markus Chimani and Finn Stutzenstein. Spanner approximations in practice. In *30th Annual European Symposium on Algorithms (ESA 2022)*, volume 244 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 37:1–37:15, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für In-